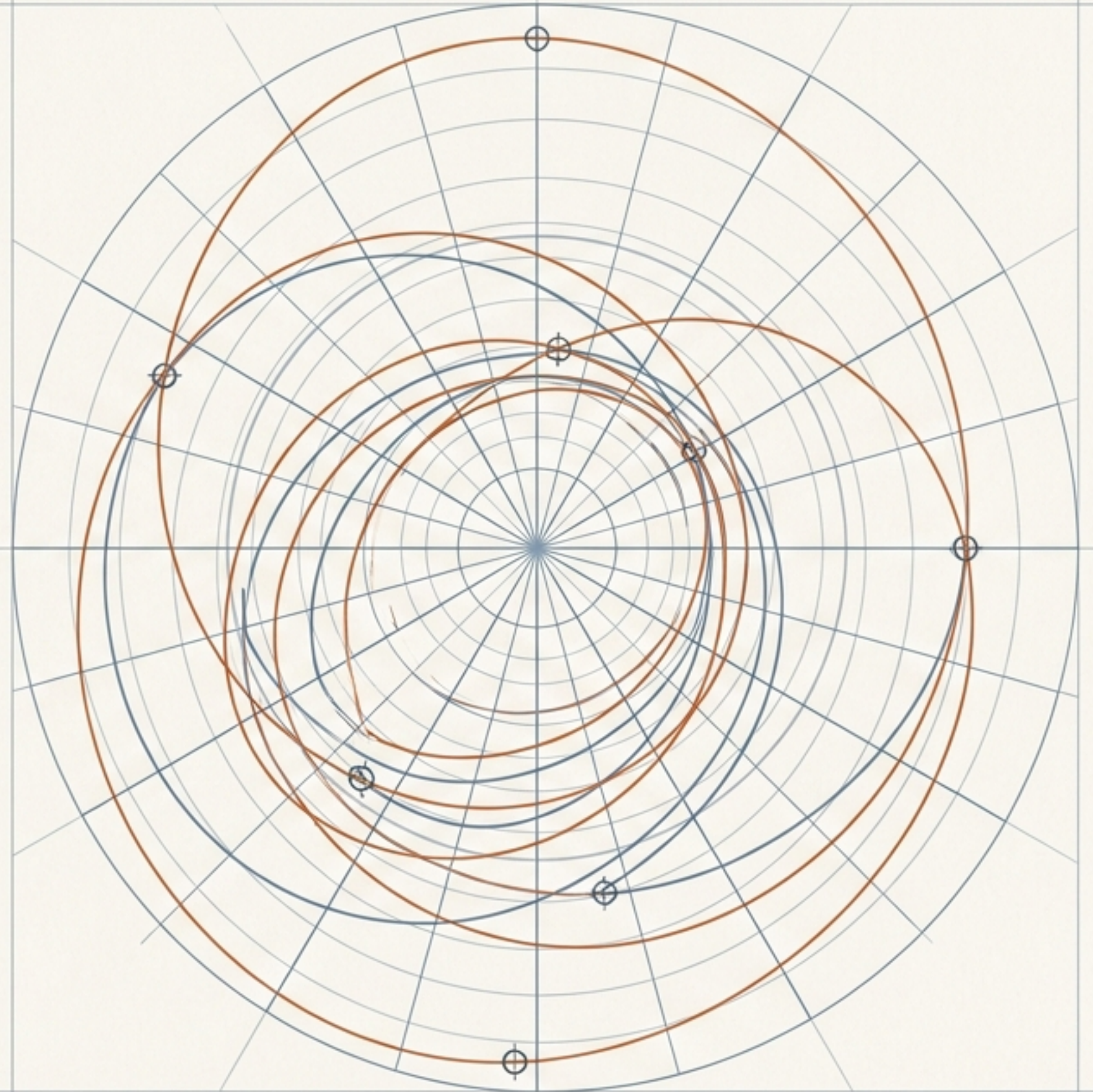


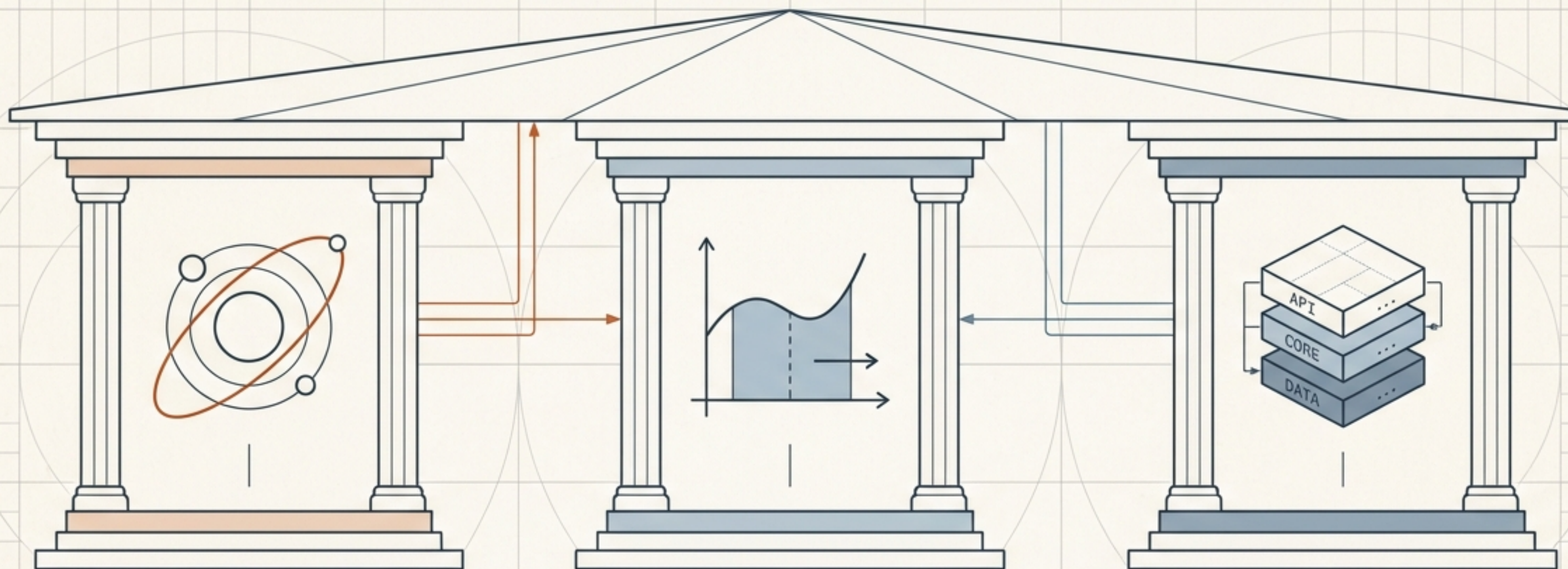
Space Systems Computing: Ampliación de Matemáticas

First Semester Milestones &
Computational Architecture

MASTER UNIVERSITARIO EN SISTEMAS
ESPACIALES // ETSIAE // SEMESTER 01



Bridging pure mathematics, orbital physics, and software engineering.



Orbital Physics

- Kepler dynamics
- N-body problems
- Lagrange points

Numerical Methods

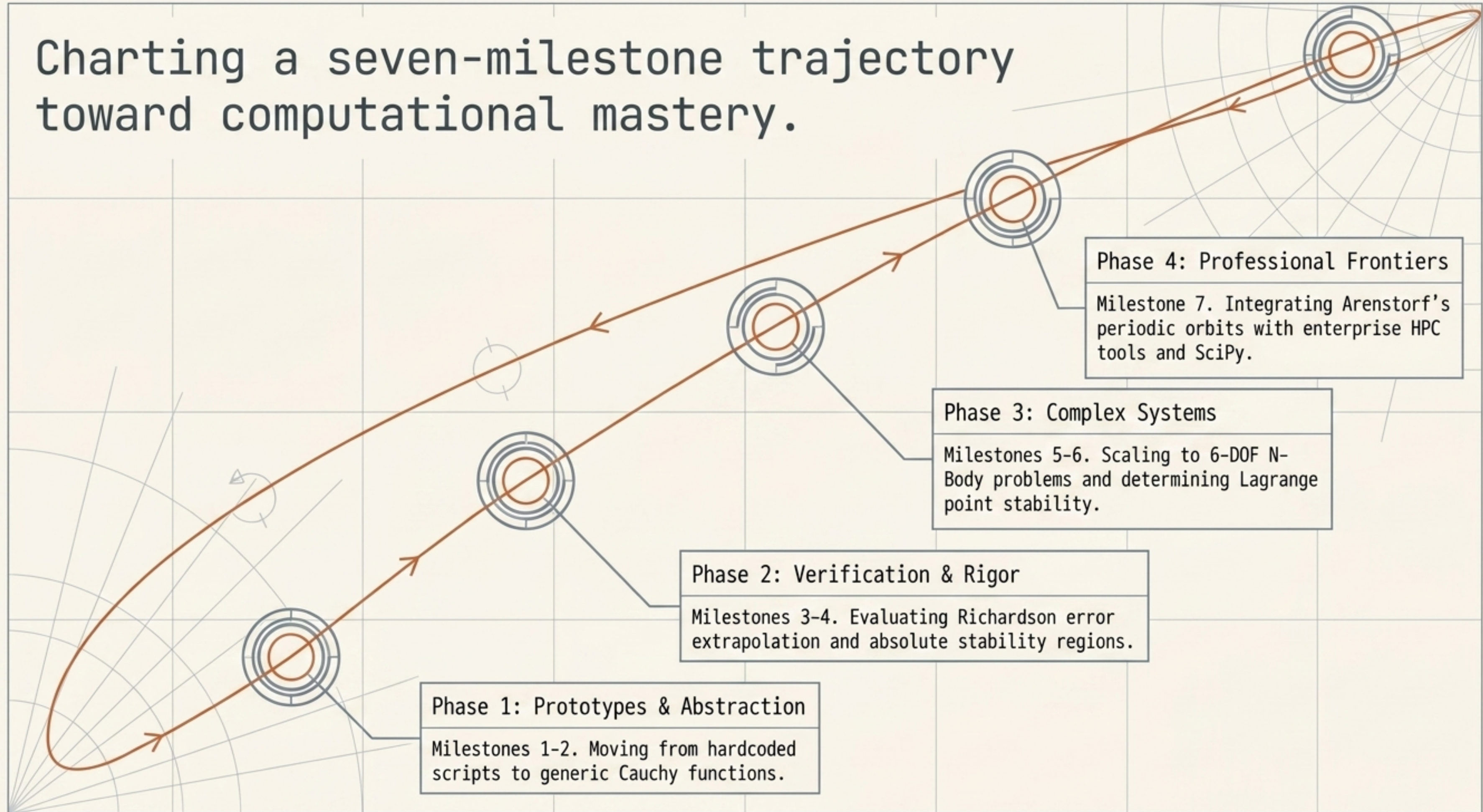
- Runge-Kutta & Crank-Nicolson
- Richardson error convergence
- Absolute stability domains

Software Engineering

- Modular Python architecture
- High-Performance Computing
- Mixed-language execution

The objective is not just to solve equations on paper, but to architect a scalable, functional space-simulation software stack from scratch.

Charting a seven-milestone trajectory toward computational mastery.



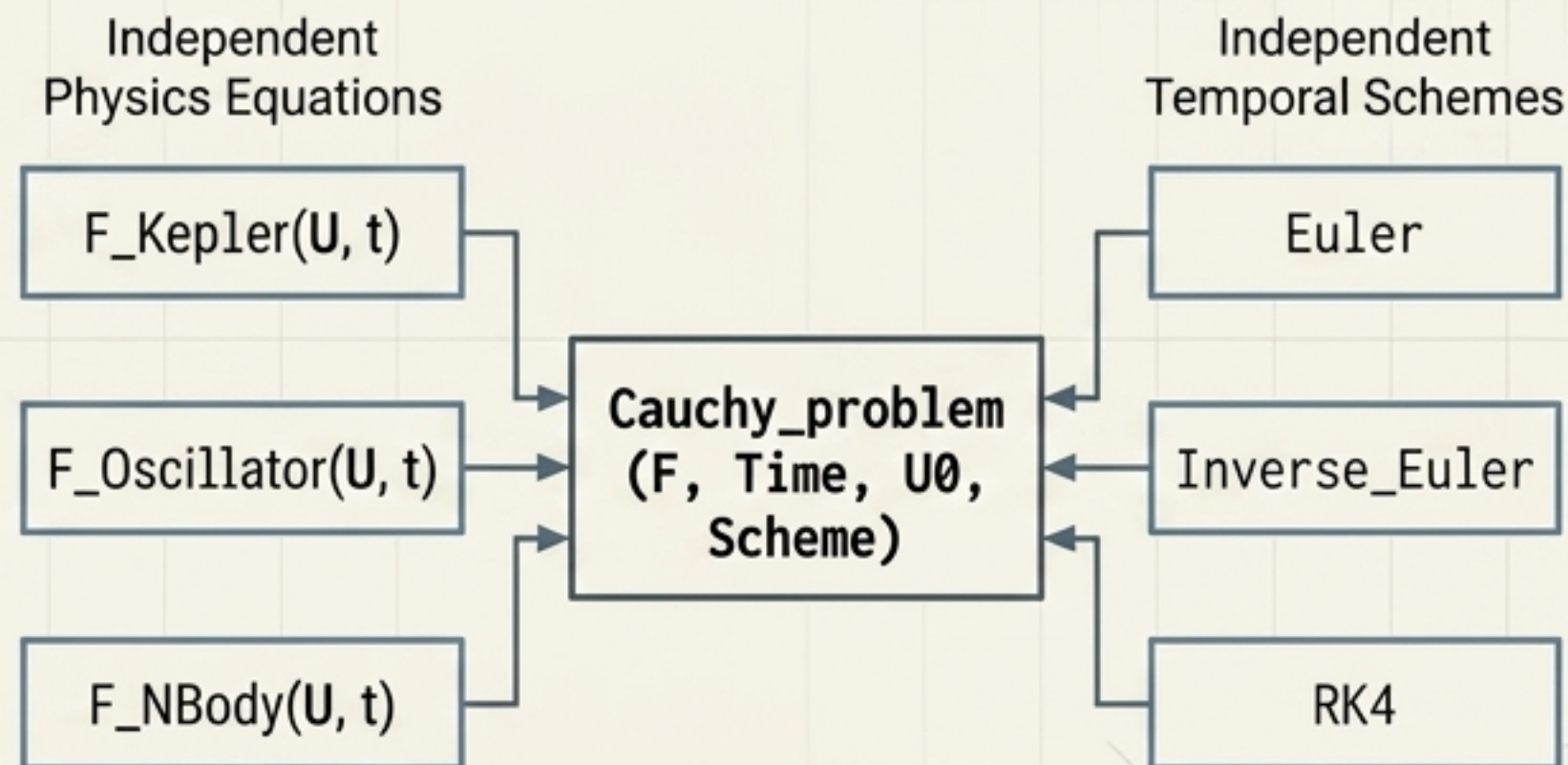
Evolving from hardcoded scripts to a scalable computational architecture

Milestone 1: Hardcoded Integration



Rigid pipeline: Only solves one physical problem with one predetermined method.

Milestone 2: Modular Architecture



Flexible abstraction: Any physics module pairs seamlessly with any temporal integration scheme.

Translating physical formalism directly into computational logic.

The Mathematical Formalism

$$\mathbf{F} = \left[\dot{\mathbf{r}}, -\frac{\mathbf{r}}{|\mathbf{r}|^3} \right]$$

Where $\mathbf{r}, \dot{\mathbf{r}} \in \mathbb{R}^2$

State Vector Definition: $\mathbf{U} = [\mathbf{r}, \dot{\mathbf{r}}]$

The Computational Implementation

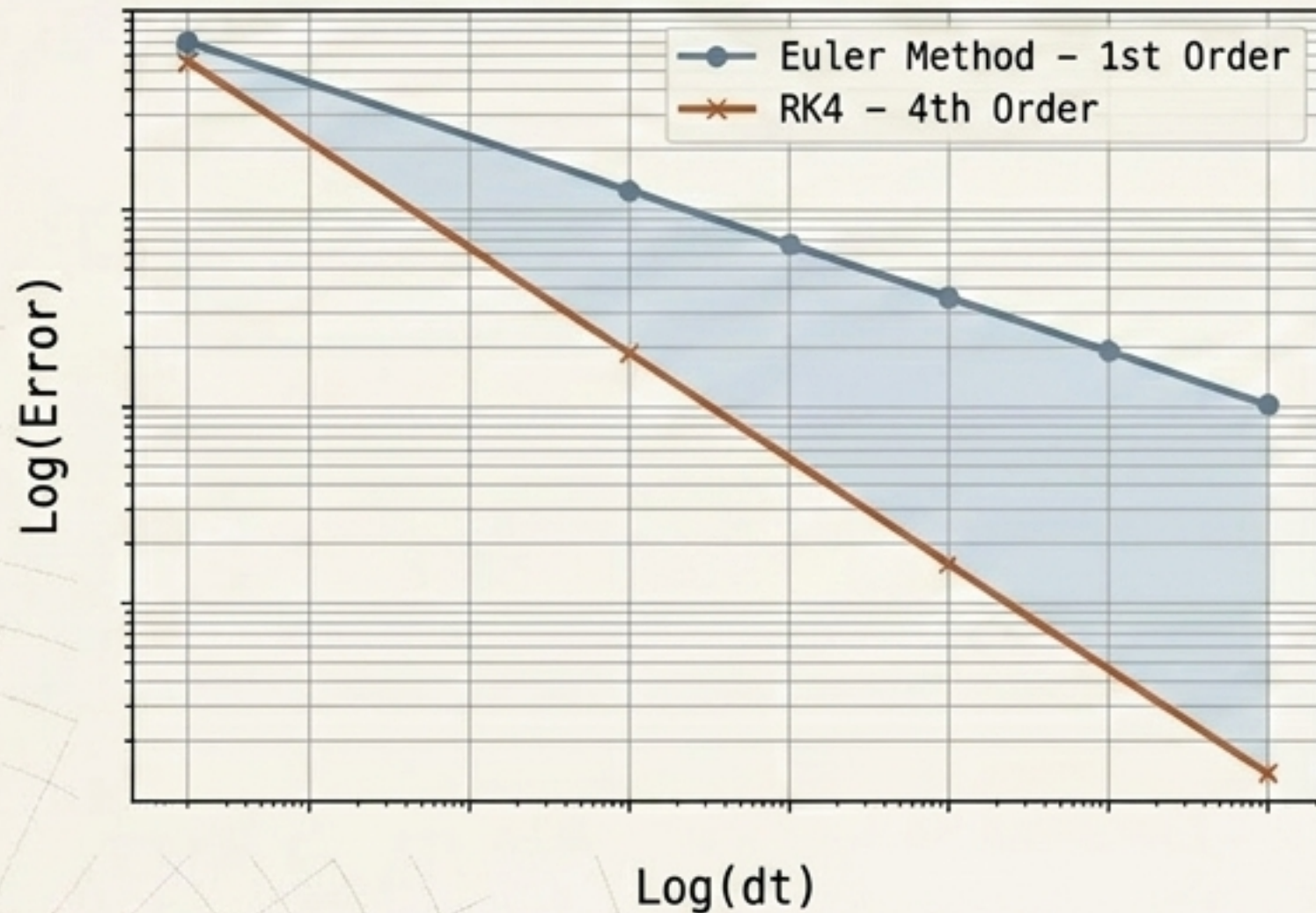
```
from ODES.Cauchy_problem import Cauchy_problem
from Physics.Orbits import Kepler

# Integrating a 4D state vector U0
Simulation(tf = 24*pi, N = 40, U0 = array([1., 0., 0., 1.]))
```

Mapping the continuous mathematical 4-dimensional state vector to a discrete computational array is the foundational interface of the entire space-simulation software stack.

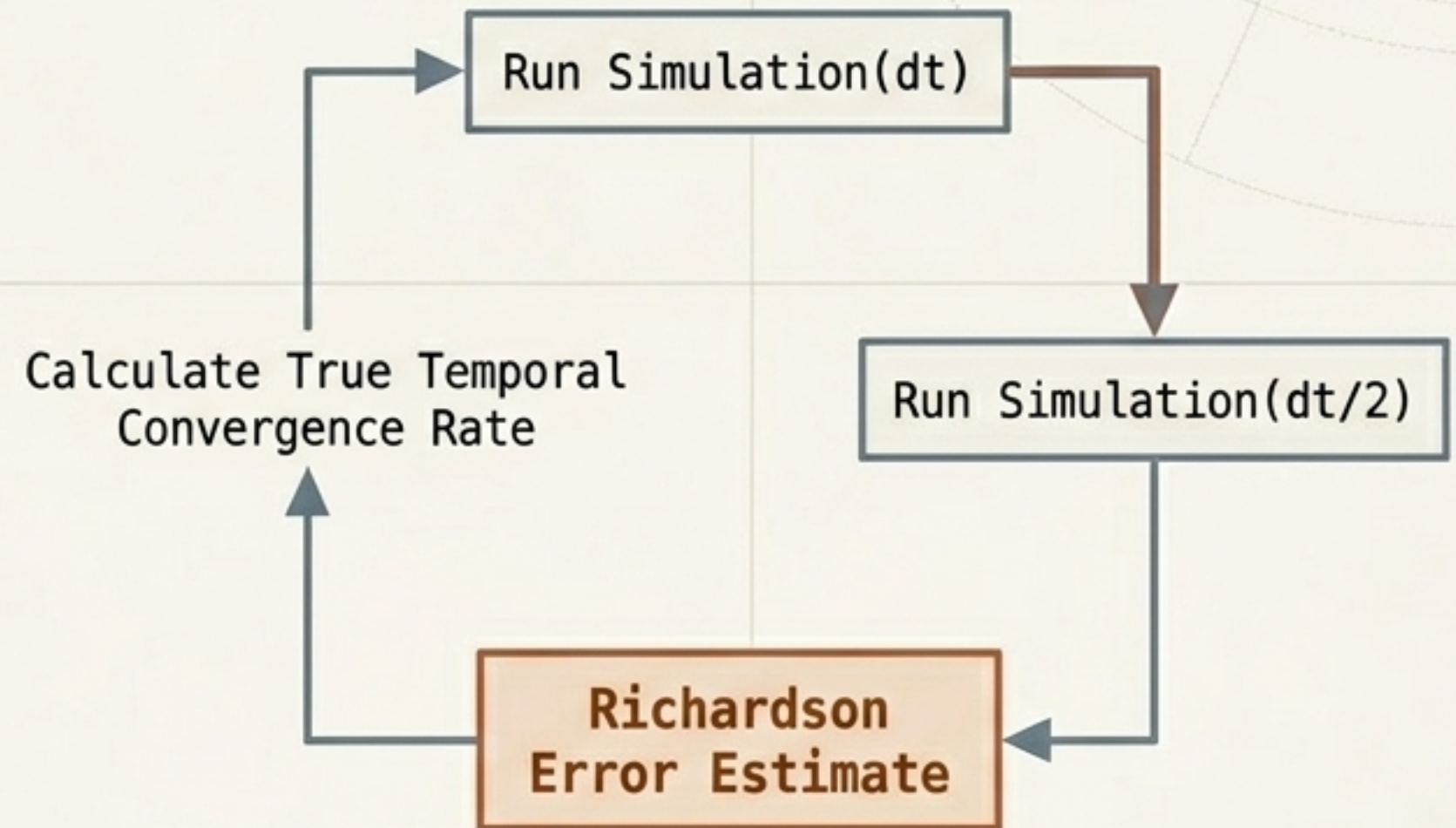
Quantifying numerical error and mapping convergence rates.

Error Convergence Graph

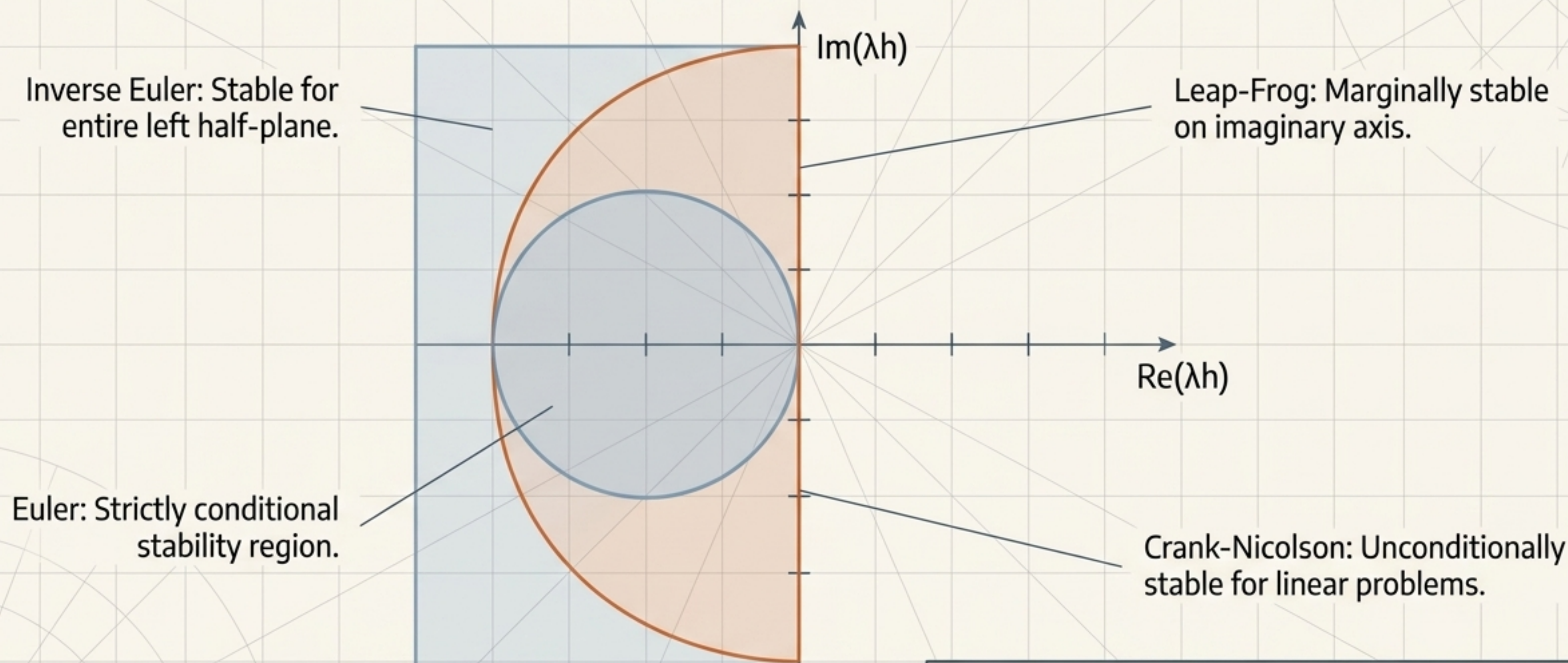


Measuring the exact computational cost required to minimize truncation errors in orbital predictions.

Richardson Extrapolation Process Loop



Determining boundaries of absolute stability using the linear oscillator.



A solver is only as reliable as its stability domain. If an orbital integration steps outside these bounds, numerical artifacts will violently shatter the physical simulation.

The Numerical Toolkit: Selecting the right temporal integration scheme.

Scheme	Type	Accuracy	Stability Profile	Primary Space Application
Euler	● Explicit	1 st Order	Poor (Conditional)	Prototyping & Basic Checks
Inverse Euler	● Implicit	1 st Order	Unconditional (Linear)	High-Damping Scenarios
Leap-Frog	● Symplectic	2 nd Order	Marginal	Energy Conservation Tracking
Crank-Nicolson	● Implicit	2 nd Order	Unconditional (Linear)	Long-Term Stable Orbits
Runge-Kutta 4	● Explicit	4 th Order	Moderate/High	General Complex Systems
Embedded RK	● Variable	Adaptive	High Precision	High-Fidelity Tracking & N-Body

Scaling computational architecture to handle the N-Body problem.

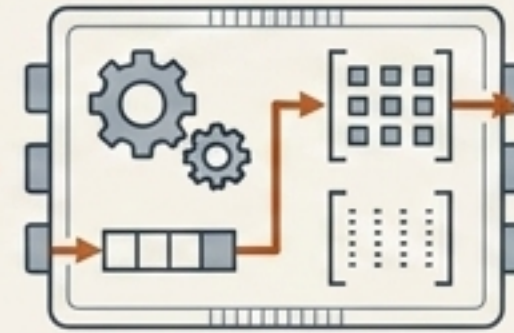
The Physics

$$\frac{d\mathbf{v}_i}{dt} = -Gm \sum_j \frac{\mathbf{r}_i - \mathbf{r}_j}{|\mathbf{r}_i - \mathbf{r}_j|^3}$$

Solving 6 degrees of freedom per body across N time steps.

Data Pipeline Schematic

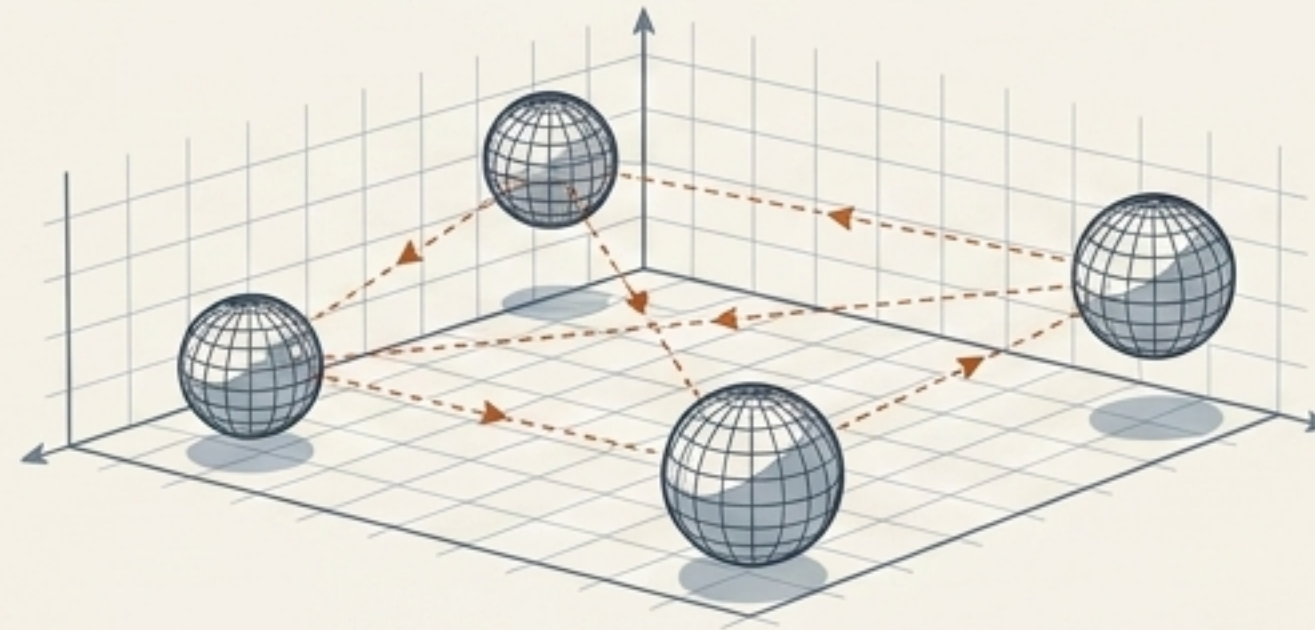
1D Raw State Vector: U



```
Us = reshape(U, (N+1, Nb, Nc, 2))
```

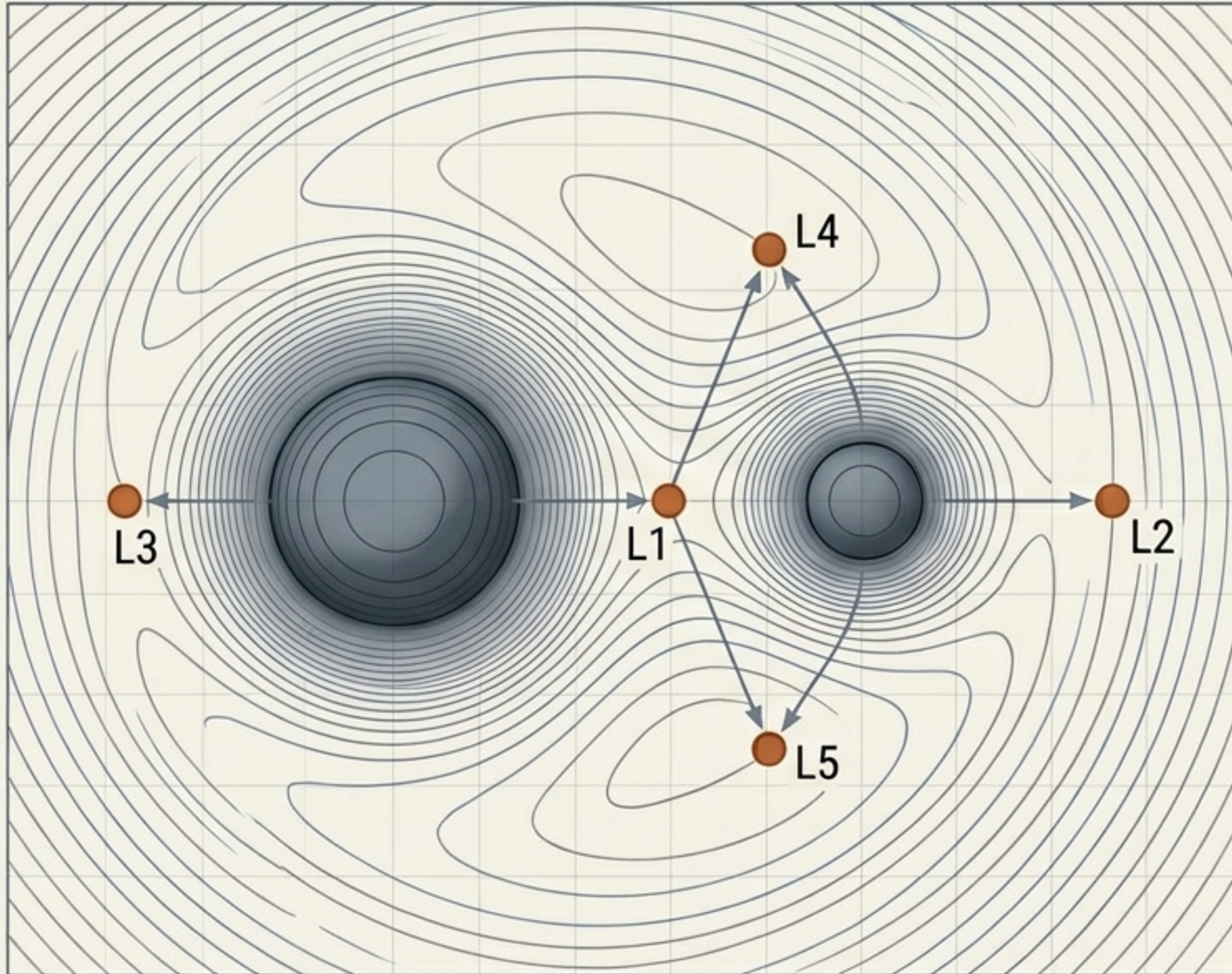
- N=1000 (Time Steps)
- Nb=4 (Gravitational Bodies)
- Nc=3 (Spatial Coordinates)

Physical Simulation



Interconnected gravitational interactions in 4-body system schematic.

Mapping orbital equilibrium in the circular restricted three-body problem.



Determining Stability via $F(U) = 0$

1. Isolate Coordinates

Define stationary spatial coordinates by setting the derivative function to absolute zero.

2. Map the Nodes

Calculate the exact spatial coordinates for all five Lagrangian points across the system.

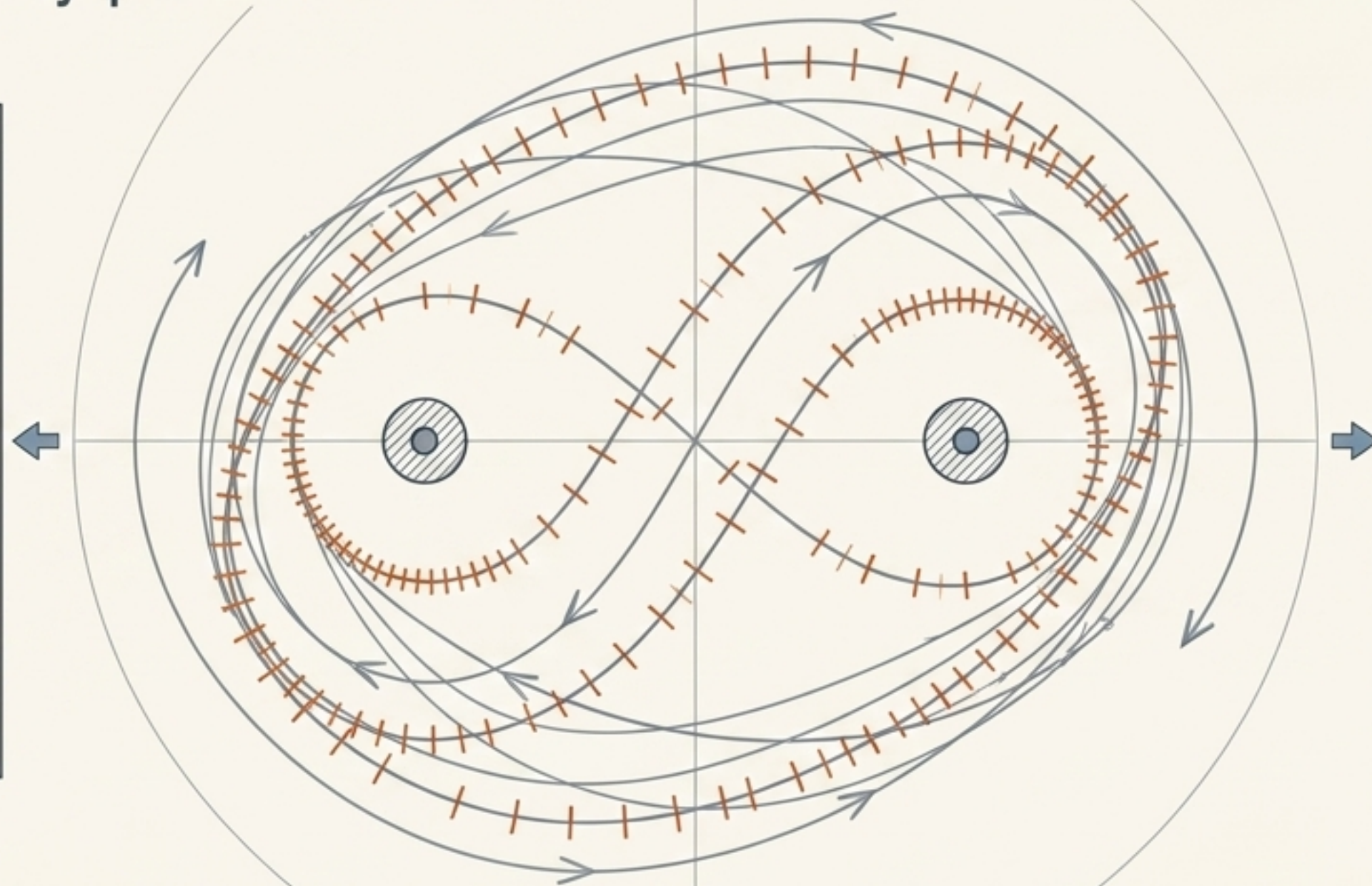
3. Deploy Embedded Solvers

Test the orbital stability of small bodies orbiting these nodes using high-order embedded Runge-Kutta schemes.

Tracking Arenstorf's periodic orbit and advanced stability parameters.

Custom Solvers

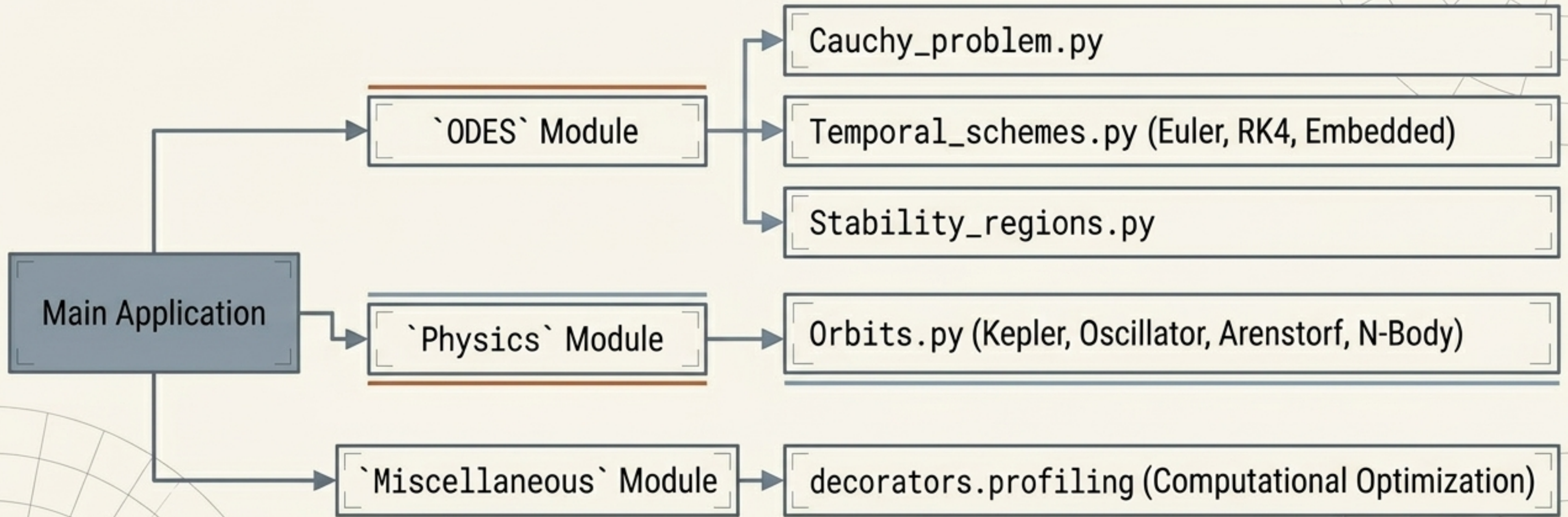
- Evaluated GBS, RK, and AM methods.
- Tight error tolerance enforcement.
- Benchmarking Lyapunov and Shampine-Gordon orbit stability.



SciPy Reference Benchmarks

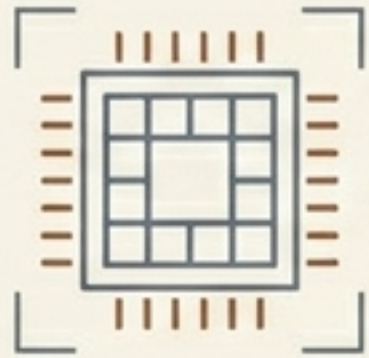
- Validated against odeint & solve_ivp.
- Compared against LSODA.
- Evaluated computational effort vs. ODEPACK & ODEX.

The final product: A production-ready orbital mechanics codebase.



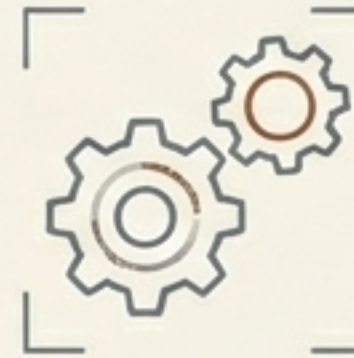
Takeaway: Students conclude the semester possessing a customized, fully modular, object-oriented library ready for advanced research and immediate execution.

Expanding the computational frontier beyond basic Python.



Parallelization

Leveraging GPU computing to handle the massive calculation overhead of the 6-DOF N-Body problem.



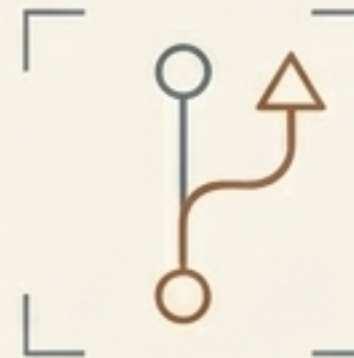
Mixed-Language Execution

Integrating Fortran, C++, and Python in a single executable structure for maximum simulation speed.



Enterprise Integration

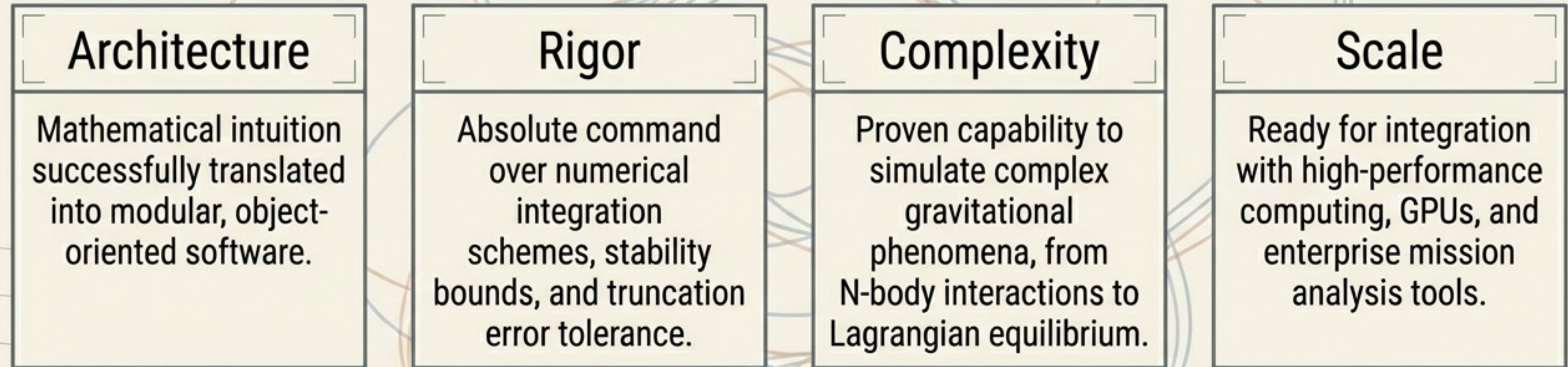
Writing custom Python scripts to interface directly with the General Mission Analysis Tool (GMAT).



CI/CD & Deployment

Implementing automatic testing with GitHub and compiling standalone Python executables.

Mission Accomplished: Core competencies established in Semester One.



SEMESTER 01 CLEAR. READY FOR ADVANCED SYSTEMS IN SEMESTER 02.